Documentação do Frontend Administrativo

Projeto HELP

Estado atual, pendências, prioridades e critérios de conclusão.

Atualizado em 20 de outubro de 2026.

github.com/annygabb/Help---TCC

Documentação do Frontend Administrativo

Projeto HELP

Estado atual, pendências, prioridades e critérios de conclusão.

Atualizado em 20 de outubro de 2026.

github.com/annygabb/Help---TCC

Guia de leitura

Este PDF apresenta, de forma humana e organizada, como o painel administrativo está hoje, o que já funciona e quais partes ainda precisam ser implementadas para o uso real.

Conteúdo principal

- Estrutura atual e recursos prontos
- Pendências detalhadas do frontend
- Módulos administrativos futuros
- Contratos esperados da API
- Prioridades, fluxos e checklist final

Pendências do Frontend Administrativo — HELP

Objetivo

Este documento descreve somente a parte visual e de experiência do painel administrativo do HELP.

Ele mostra:

- o que já existe no frontend;
- como cada tela funciona hoje;
- o que ainda precisa ser acrescentado;
- o que depende do backend para funcionar;
- como o fluxo deve ficar quando estiver completo;
- a ordem recomendada para desenvolver as próximas etapas.

Nenhuma tela foi modificada durante a criação deste documento. Ele serve como guia para as próximas implementações.

Resumo humano

O painel administrativo já tem uma boa base visual. Ele possui menu lateral, página inicial com indicadores, listagem de usuários, listagem de empresas, filtros, paginação, telas de detalhes e botões para ativar ou desativar contas.

A maior parte das telas já está desenhada para receber dados reais. O que falta é completar os comportamentos da interface, melhorar o retorno ao administrador e conectar tudo aos endpoints administrativos do backend.

Hoje o painel é parecido com uma recepção pronta, com balcão, placas e formulários, mas ainda aguardando alguns serviços internos. As páginas existem, porém ações como consultar uma conta administrativa real, desativar uma pessoa com justificativa e visualizar um histórico confiável dependem do backend.

Estrutura atual do frontend

Arquivo: AdminRoute.jsx
Responsabilidade atual: Verifica se existe token e se a conta retornada por /auth/me é do tipo ADMIN

Arquivo: AdminLayout.jsx
Responsabilidade atual: Exibe menu lateral, cabeçalho, identificação do administrador e botão de saída

Arquivo: AdminHome.jsx
Responsabilidade atual: Exibe cartões com os números gerais da plataforma

Arquivo: AdminUsuarios.jsx
Responsabilidade atual: Lista usuários, aplica busca e filtros, pagina e altera status

Arquivo: AdminUsuarioDetalhe.jsx
Responsabilidade atual: Mostra perfil, conexões e histórico de uma conta

Arquivo: AdminEmpresas.jsx
Responsabilidade atual: Lista empresas, aplica busca e filtro de status, pagina e altera status

Arquivo: AdminEmpresaDetalhe.jsx
Responsabilidade atual: Mostra dados corporativos e histórico da empresa

Arquivo: adminComponentes.jsx
Responsabilidade atual: Reúne badges, paginação, avisos, formatação de dados e debounce

Arquivo: adminService.js
Responsabilidade atual: Centraliza as chamadas da API administrativa

Arquivo: Admin.css
Responsabilidade atual: Controla o visual e a responsividade do painel

O que já está pronto

Navegação administrativa

O painel já possui:

- rota principal /admin;
- rota de usuários;
- rota de detalhes do usuário;
- rota de empresas;
- rota de detalhes da empresa;
- menu lateral responsivo;
- identificação da conta administrativa;
- encerramento da sessão.

Verificação inicial de acesso

O componente AdminRoute já:

- 1. verifica se há token no navegador;
- 2. consulta GET /auth/me;
- 3. libera o painel quando tipoConta é ADMIN;
- 4. redireciona para o login quando não existe sessão;
- 5. impede visualmente a entrada de contas sem permissão.

Essa verificação melhora a experiência, mas a segurança definitiva precisa continuar no backend.

Visão geral

A página inicial já tem cartões preparados para:

- contas de pessoas;
- candidatos;
- administradores;

- empresas;
- contas desativadas;
- empresas desativadas.

Gestão de usuários

A tela já oferece:

- busca por nome ou e-mail;
- filtro por candidato ou administrador;
- filtro por conta ativa ou desativada;
- debounce para evitar requisições a cada tecla;
- paginação;
- botão de detalhes;
- botão de ativação ou desativação;
- bloqueio visual para a própria conta;
- mensagens de sucesso e falha;
- estados de carregamento e lista vazia.

Gestão de empresas

A tela já oferece:

- busca por razão social, CNPJ ou e-mail;
- filtro de status;
- paginação;
- botão de detalhes;
- ativação e desativação;
- mensagens de retorno;
- estado vazio e carregamento.

Telas de detalhes

O detalhe do usuário já está preparado para receber:

- nome;
- e-mail;
- tipo da conta;
- status;
- cargo;
- localização;
- CPF mascarado;
- data de cadastro;
- número de seguidores;
- número de pessoas seguidas;
- histórico administrativo.

O detalhe da empresa já está preparado para receber:

- razão social;
- CNPJ;
- e-mail;
- status;
- data de cadastro;
- histórico administrativo.

O que falta no frontend

1. Página própria de acesso negado

Como está hoje

Quando uma pessoa autenticada não é administradora, ela é redirecionada para a página inicial.
A interface não explica o motivo.

Como deve ficar

Criar uma página como /acesso-negado contendo:

- título informando que a área é restrita;
- explicação curta;
- botão para voltar ao sistema;
- botão para entrar com outra conta;
- identificação visual consistente com o painel.

Benefício

A pessoa entende o que aconteceu e não interpreta o redirecionamento como erro do site.

---

2. Tratamento global de sessão expirada

Como está hoje

Cada tela trata a falha da API separadamente. Uma resposta 401 pode apenas exibir uma mensagem, sem encerrar a sessão inválida.

Como deve ficar

Criar um tratamento central no serviço da API:

- 1. detectar 401;
- 2. remover token e dados locais;
- 3. informar que a sessão expirou;
- 4. encaminhar para o login;
- 5. preservar, quando possível, a rota para retorno após nova autenticação.

Para 403, o frontend deve encaminhar à página de acesso negado sem apagar uma sessão válida.

Benefício

Evita que o administrador permaneça em uma tela que não consegue mais carregar dados.

---

3. Modal de ativação e desativação

Como está hoje

As telas usam window.confirm. Essa confirmação não coleta justificativa, tem pouca informação e não acompanha o padrão visual do sistema.

Como deve ficar

Criar um componente reutilizável, por exemplo AdminStatusDialog, com:

- nome da conta ou empresa;

- ação que será executada;
- resumo do impacto;
- campo obrigatório de justificativa;
- botão cancelar;
- botão confirmar;
- bloqueio enquanto a solicitação estiver em andamento;
- foco controlado para teclado;
- mensagem clara se a API recusar a ação.

Dados esperados

```
{
"ativo": false,
"motivo": "Solicitação do responsável pela conta"
}
```

Benefício

A ação fica mais segura, compreensível e preparada para auditoria.

---

4. Filtros preservados na URL

Como está hoje

Busca, tipo, status e página ficam somente no estado do componente. Ao atualizar a página ou voltar dos detalhes, os filtros podem ser perdidos.

Como deve ficar

Usar parâmetros na URL.

Exemplo:

```
#/admin/usuarios?busca=anny&tipo=CANDIDATO&status=ativo&page=0
```

Para empresas:

```
#/admin/empresas?busca=tecnologia&status=ativo&page=1
```

Benefício

O administrador pode atualizar a página, voltar dos detalhes e compartilhar uma consulta sem perder o contexto.

---

5. Ordenação das tabelas

Como está hoje

As tabelas não mostram controle de ordenação.

Como deve ficar

Permitir ordenar por:

- nome ou razão social;
- data de cadastro;

- status;
- tipo de conta, quando aplicável.

O cabeçalho deve indicar visualmente a direção crescente ou decrescente.

Dependência

O backend precisa aceitar parâmetros como:

```
sort=nome,asc
sort=criadoEm,desc
```

6. Botão de tentar novamente

Como está hoje

Quando ocorre uma falha de rede, a tela apresenta uma mensagem, mas o administrador precisa atualizar a página manualmente.

Como deve ficar

Os avisos de falha devem incluir um botão Tentar novamente que repita somente a requisição necessária.

Benefício

Melhora a recuperação de falhas temporárias do Railway, internet ou banco.

7. Carregamento visual estável

Como está hoje

As tabelas mostram apenas o texto Carregando....

Como deve ficar

Usar linhas de carregamento no mesmo tamanho da tabela final, mantendo:

- cabeçalho visível;
- espaço reservado;
- quantidade aproximada de linhas;
- botões temporariamente indisponíveis.

Benefício

Evita mudanças bruscas no layout e deixa a interface mais profissional.

8. Mensagens padronizadas

Como está hoje

Existem mensagens de erro e sucesso, mas elas aparecem de formas diferentes dependendo da tela.

Como deve ficar

Criar um padrão único com:

- sucesso;
- atenção;
- erro;
- informação;
- opção de fechar;
- desaparecimento automático somente para mensagens não críticas;
- aria-live para leitores de tela.

Exemplos

- “Conta desativada com sucesso.”
- “A empresa continua ativa porque a operação não foi concluída.”
- “Sua sessão expirou. Entre novamente.”
- “Você não possui permissão para realizar esta ação.”

---

9. Detalhes completos do usuário

Como está hoje

A tela mostra informações básicas, conexões e histórico.

Como deve ficar

Acrescentar, quando o backend disponibilizar:

- foto de perfil;
- biografia;
- cargo atual;
- experiências profissionais;
- cursos e certificações;
- data da última atividade;
- quantidade de publicações;
- quantidade de mensagens ou denúncias apenas quando houver justificativa administrativa;
- situação da conta;
- motivo da última desativação.

Os dados devem ser apenas para consulta. O administrador não deve editar o perfil fingindo ser o usuário.

---

10. Detalhes completos da empresa

Como está hoje

A tela mostra razão social, CNPJ, e-mail, cadastro, status e histórico.

Como deve ficar

Acrescentar:

- nome do responsável;
- telefone de contato, quando permitido;
- quantidade de vagas;
- vagas ativas;
- vagas encerradas;

- última atividade;
- motivo da última desativação;
- impacto da desativação nas vagas publicadas.

A interface deve explicar se desativar uma empresa também oculta suas vagas.

11. Prevenção de ações perigosas

O frontend deve bloquear visualmente:

- desativação da própria conta;
- desativação do último administrador ativo;
- cliques repetidos;
- envio sem justificativa;
- nova tentativa enquanto a anterior ainda está em andamento.

Mesmo com esses bloqueios no frontend, o backend precisa repetir todas as validações.

12. Atualização dos indicadores

Como está hoje

Os cartões da visão geral são carregados quando a página abre.

Como deve ficar

Adicionar:

- horário da última atualização;
- botão para atualizar;
- carregamento individual dos cartões;
- mensagem quando algum indicador estiver indisponível;
- período de referência quando futuramente houver filtros por data.

13. Estado vazio mais orientativo

Como está hoje

As telas informam que nenhum registro foi encontrado.

Como deve ficar

Diferenciar:

- banco sem registros;
- filtro sem resultado;
- busca sem correspondência;
- falha de carregamento.

Exemplo de busca vazia:

> Nenhuma conta corresponde aos filtros escolhidos. Limpe os filtros ou tente outro termo.

14. Responsividade e acessibilidade

Antes de considerar o painel pronto, revisar:

- tabela no celular;
- rolagem horizontal restrita somente ao conteúdo necessário;
- tamanho mínimo dos botões;
- foco visível;
- navegação completa por teclado;
- fechamento do modal pela tecla Esc;
- retorno do foco ao botão que abriu o modal;
- rótulos em todos os campos;
- contraste de badges e mensagens;
- aria-live nos avisos;
- títulos em ordem correta;
- respeito a prefers-reduced-motion.

15. Testes do frontend administrativo

Devem existir testes para:

Controle de acesso

- sem token;
- token expirado;
- usuário comum;
- empresa;
- administrador;
- resposta 401;
- resposta 403.

Listagens

- carregamento;
- lista vazia;
- busca;
- filtros;
- paginação;
- ordenação;
- restauração dos filtros pela URL;
- falha e nova tentativa.

Alteração de status

- abertura do modal;
- justificativa obrigatória;
- cancelamento;
- confirmação;
- bloqueio da própria conta;
- sucesso;
- falha;
- prevenção de clique repetido.

Detalhes

- registro existente;
- registro inexistente;
- histórico vazio;
- histórico preenchido;

- campos opcionais ausentes.

Novos módulos administrativos ainda não existentes

As telas atuais cobrem somente visão geral, usuários e empresas. Futuramente podem ser criadas áreas para:

Módulo: Vagas
Funções principais: Consultar, suspender, encerrar e revisar vagas

Módulo: Cursos
Funções principais: Criar, editar, publicar, desativar e acompanhar matrículas

Módulo: Matrículas
Funções principais: Consultar status, token, curso e situação do acesso

Módulo: Pagamentos
Funções principais: Consultar transações, status, falhas e estornos

Módulo: Publicações
Funções principais: Moderar conteúdos denunciados

Módulo: Denúncias
Funções principais: Analisar motivo, evidências, responsável e decisão

Módulo: E-mails
Funções principais: Acompanhar falhas de SMTP e reenvios

Módulo: Auditoria
Funções principais: Consultar todas as ações administrativas

Módulo: Configurações
Funções principais: Regras gerais da plataforma e integrações

Esses módulos não devem ser apenas telas vazias. Cada um precisa de endpoints, permissões e regras definidas no backend.

Contratos que o frontend já espera

Método: GET
Endpoint: /auth/me
Uso: Validar a conta administrativa

Método: GET
Endpoint: /admin/resumo
Uso: Carregar os indicadores

Método: GET
Endpoint: /admin/contas
Uso: Listar usuários

Método: GET
Endpoint: /admin/contas/{id}
Uso: Abrir detalhes do usuário

Método: PATCH
Endpoint: /admin/contas/{id}/status
Uso: Ativar ou desativar usuário

Método: GET
Endpoint: /admin/empresas
Uso: Listar empresas

Método: GET
Endpoint: /admin/empresas/{id}
Uso: Abrir detalhes da empresa

Método: PATCH
Endpoint: /admin/empresas/{id}/status
Uso: Ativar ou desativar empresa

Formato esperado para paginação

```
{
"content": [],
"number": 0,
"totalPages": 1,
"totalElements": 0
}
```

Formato recomendado para erros

```
{
"codigo": "CONTA_NAO_ENCONTRADA",
"mensagem": "A conta solicitada não foi encontrada."
}
```

Arquivos sugeridos para a próxima implementação

Arquivo sugerido: AdminAccessDenied.jsx
Finalidade: Página de acesso negado

Arquivo sugerido: AdminStatusDialog.jsx
Finalidade: Modal de ativação e desativação

Arquivo sugerido: AdminFeedback.jsx
Finalidade: Avisos padronizados

Arquivo sugerido: AdminTableSkeleton.jsx
Finalidade: Carregamento das tabelas

Arquivo sugerido: useAdminQueryParams.js
Finalidade: Sincronização de filtros com a URL

Arquivo sugerido: useAdminSession.js
Finalidade: Tratamento de sessão e permissões

Arquivo sugerido: adminApiInterceptor.js
Finalidade: Respostas globais 401 e 403

Os nomes são sugestões. A implementação deve seguir o padrão de organização que já existe no projeto.

Prioridades recomendadas

Prioridade 0 — necessária para uso real

- página de acesso negado;

- sessão expirada;
- endpoints administrativos funcionando;
- autorização validada pelo backend;
- modal com justificativa;
- bloqueio de ações repetidas;
- mensagens consistentes.

Prioridade 1 — operação diária

- filtros na URL;
- botão tentar novamente;
- ordenação;
- carregamento estável;
- detalhes completos;
- última atualização dos indicadores;
- testes dos fluxos principais.

Prioridade 2 — expansão

- vagas;
- cursos;
- matrículas;
- pagamentos;
- denúncias;
- moderação;
- auditoria geral;
- configurações.

Como o fluxo deve funcionar no final

Entrada no painel

- 1. A pessoa abre /admin.
- 2. O frontend verifica a sessão.
- 3. O backend confirma identidade, tipo e status.
- 4. Administrador ativo entra.
- 5. Conta sem permissão vê a página de acesso negado.
- 6. Sessão expirada volta ao login com mensagem explicativa.

Desativação de conta

- 1. O administrador localiza a conta.
- 2. Abre a confirmação.
- 3. Lê o impacto.
- 4. Informa a justificativa.
- 5. Confirma.
- 6. O botão entra em carregamento.
- 7. O backend valida a permissão e executa a ação.
- 8. A interface atualiza a linha.
- 9. O histórico mostra responsável, data, ação e motivo.

Retorno dos detalhes

- 1. O administrador abre um registro.
- 2. Consulta os dados.
- 3. Volta para a listagem.

- 4. Busca, filtros, página e ordenação permanecem iguais.

Checklist de conclusão do frontend

Acesso

- [ ] Existe página de acesso negado.
- [ ] 401 encerra a sessão inválida.
- [ ] 403 mantém a sessão e mostra falta de permissão.
- [ ] Conta desativada recebe orientação clara.

Navegação

- [ ] Filtros permanecem na URL.
- [ ] Voltar dos detalhes preserva a consulta.
- [ ] Menu funciona por teclado e celular.

Ações

- [ ] Ativação e desativação usam modal.
- [ ] Justificativa é obrigatória.
- [ ] O impacto da ação fica visível.
- [ ] O botão bloqueia durante o envio.
- [ ] Não há envio duplicado.

Listagens

- [ ] Busca funciona.
- [ ] Filtros funcionam.
- [ ] Paginação funciona.
- [ ] Ordenação funciona.
- [ ] Existe botão para tentar novamente.
- [ ] Carregamento não desloca o layout.

Detalhes

- [ ] Usuário possui visão completa.
- [ ] Empresa mostra vagas e responsável.
- [ ] Histórico possui autor, data, ação e motivo.
- [ ] Dados sensíveis continuam protegidos.

Qualidade

- [ ] Fluxos principais possuem testes.
- [ ] Interface funciona por teclado.
- [ ] Mensagens são acessíveis.
- [ ] Layout funciona no celular.
- [ ] Não há dados administrativos fixos no código.

Resultado esperado

Quando essas etapas forem concluídas, o painel deixará de ser apenas uma estrutura visual preparada e se tornará uma área administrativa completa, compreensível e segura para o uso diário.

O administrador conseguirá entender o estado da plataforma, localizar contas, analisar informações, executar ações justificadas e acompanhar tudo pelo histórico, sem precisar acessar diretamente o banco de dados.